TRANSITIVE RELATIONSHIPS

Cedar policies can leverage transitive relationships in the entity store. If a relationship path connects a principal to a resource—regardless of how many edges it involves—Cedar can follow it during evaluation.

For example, ACME could extend the “manager of owner” rule to include indirect managers:

```
permit(
    principal in ACME::Employee,
    action == ACME::"doc:view",
    resource in ACME::Document
)
when { resource.owner.manager.manager == principal };
```

This policy explicitly follows transitive chains in the graph. One way to avoid writing out `manager.manager.manager` chains is to model your organization’s management tree as a true hierarchy in the entity store. If each employee’s parent is their immediate manager, Cedar’s `in` operator will follow that chain transitively—so `principal in resource.owner` will match not only the owner but also any manager up the chain.

This works anywhere `in` is used, without additional policy code. If your management structure isn’t a strict hierarchy, you could pre-compute a managers team for each employee in the entity store and check membership there.

The main point is that the power (and risk) of transitive relationships depends on the entity store; if the graph encodes multi-hop connections, Cedar will follow them. This makes it easy to model real-world hierarchies, but also means that global constraints should account for the entire reach of these relationships.